

fastglmm: Fitting Massive Generalized Linear Mixed Models in R

Gabriel E. Hoffman

Icahn School of Medicine at Mount Sinai
James J. Peters VA Medical Center

Panos Roussos

Icahn School of Medicine at Mount Sinai
James J. Peters VA Medical Center

Abstract

Mixed models are a family of statistical methods for grouped or repeated measures data. Linear mixed models (LMM) consider responses with normal errors and generalized linear mixed models (GLMM) extend this framework to non-normal responses. Although widely applied across many fields, parameter estimation and hypothesis testing is computationally demanding for large datasets. We consider a special case of particular interest, a weighted LMM with a single random effect, and present an efficient algorithm for parameter estimation on thousands of samples. We extend this to the case of GLMMs using penalized quasi-likelihood. We have implemented these methods in the **fastglmm** package for R using a C++ backend for efficiency. The package applies familiar mixed model syntax and R generics for ease-of-use.

Keywords: generalized linear mixed models, penalized quasi-likelihood, R, spectral decomposition.

1. Introduction

Mixed models have a long history and are widely used in statistical analysis for modeling grouped or repeated measures data. Fitting linear mixed models can be computationally demanding and widely used software apply a range of approaches for parameter estimation. The `lme()` function in **nlme** models a broad range of covariance structures using generalized least squares but is computationally demanding (Pinheiro and Bates 2000). The **mmrm** package also fits a broad range of covariances (Krieger, Li, Dedic, Kelkhoff, Kunzmann, Lang, Stock, Wang, James, Sidi, Leibovitz, Sjoberg, and Ivan 2026). The `lmer()` function in **lme4** uses sparse matrix methods and efficient linear algebra for penalized least squares to speed up computation for a large subset of mixed models (Bates, Mächler, Bolker, and Walker 2015).

Fitting non-normal responses with generalized linear mixed models poses an additional challenge of approximating a high dimensional integral involving the random effects. The `glmer()` function in **lme4** use a Laplace approximation to evaluate the high dimensional integral. The `glmmPQL()` function in **MASS** uses penalized quasi-likelihood and iteratively calls `lme()` from **nlme** (Venables and Ripley 2002). **NBZIMM** extends the implementation of `glmmPQL()` to

the negative binomial model (Zhang and Yi 2020). **nebula** uses a large-sample approximation to the negative binomial mixed model (He, Davila-Velderrain, Sumida, Hafler, Kellis, and Kulinski 2021). **GLMMadaptive** uses adaptive Gaussian quadrature (Rizopoulos 2025). The `glmmTMB()` function in **glmmTMB** uses the Laplace approximation and automatic differentiation to fit complex covariance structures and zero-inflated models (McGillicuddy, Popovic, Bolker, and Warton 2025).

Due to the computational expense of fitting these models on large datasets, fast algorithms for specific linear mixed models have been developed by the genetics community (Zhou and Stephens 2012; Lippert, Listgarten, Liu, Kadie, Davidson, and Heckerman 2011; Kang, Sul, Service, Zaitlen, Kong, Freimer, Sabatti, and Eskin 2010), and more recently, groups have developed generalized linear mixed models (Zhou, Zhao, Nielsen, Fritsche, LeFaive, Gagliano Taliun, Bi, Gabrielsen, Daly, Neale, Hveem, Abecasis, Willer, and Lee 2020). But these tools are not suitable for use within R workflows since they are either command line tools, or R packages targeted at specific datatypes.

By considering a special case of the LMM of particular interest, we develop an efficient algorithm to accelerate parameter estimation for a weighted LMM with a single random effect. We then extend this to GLMMs using penalized quasi-likelihood (Breslow and Clayton 1993).

Here we present the **fastglmm** R package to efficiently fit LMMs and GLMMs on massive datasets. The package applies familiar mixed model syntax and R generics for ease-of-use, and includes functionality for parameter estimation, hypothesis testing, and variance partitioning analysis.

The article is organized as follows. In Section 2, we give motivating examples using the **fastglmm** software. In Section 3, we describe the interface and R generics to extract results from the model fit. In Section 4, we describe efficient methods used to fit LMMs and GLMMs. In Section 5, we consider summarizing the model fit using hypothesis testing and variance partitioning analysis. In Section 6 we discuss the implementation. In Section 7 we compare performance to other packages. Section 8 contains the conclusion and future directions.

2. Motivating examples

Our **fastglmm** package is designed to be intuitive for users of other mixed model software in R. To demonstrate the interface, consider the **sleepstudy** dataset distributed with **lme4** of average reaction time on each day for subjects in a sleep deprivation study (Belenky, Wesensten, Thorne, Thomas, Sing, Redmond, Russo, and Balkin 2003). Let **Reaction** time be the average reaction time in milliseconds, **Days** the number of days with sleep deprivation, and **Subject** being the subject identifier. Using this repeated measures data, we can fit the reaction time as a function of the days of sleep deprivation while modeling subject as a random effect using the `fastlmm()` function:

```
library(fastglmm)
fit <- fastlmm(Reaction ~ Days + (1 | Subject), lme4::sleepstudy)
```

```

fit
#>
#> Call:
#> fastlmm(formula = Reaction ~ Days + (1 | Subject), data = lme4::sleepstudy)
#>
#> Coefficients:
#> (Intercept)      Days
#>      251.41      10.47

summary(fit)
#> Linear mixed model fit by ML ['fastlmm']
#> Formula: Reaction ~ Days + (1 | Subject)
#>
#> Coefficients:
#>              Estimate Std. Error   ddf t value Pr(>|t|)
#> (Intercept) 251.4051      9.5062  24.5  26.45  <2e-16 ***
#> Days        10.4673      0.8017 162.0  13.06  <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual df: 162.2
#>
#> Variance components:
#>   sigSq_g: 1297
#>   sigSq_e: 954.5
#>   delta:  0.736

```

Estimates of the fixed effects, their standard errors, and random effects using `fastlmm()` are identical to the results from `lmer(..., REML=FALSE)` using maximum likelihood rather than REML. Here, `sigSq_g` is the variance of the random effect, `sigSq_e` is the residual variance, and `delta` is the ratio.

2.1. Negative binomial mixed model

Recent advances in single cell transcriptomics have enabled generation of large-scale datasets of millions of cells from hundreds or thousands of samples (Rood, Wynne, Robson, Hupalowska, Randell, Teichmann, and Regev 2025). In differential expression analysis, the goal is to identify genes whose expression is associated with a variable of interest. Single cell transcriptomics produces counts for each gene and each observed cell. To be concrete, consider counts from the gene `PTPRG` from 249 subjects measured across 60k microglia cells where an average of 202 cells were observed per subject (Hoffman, Lee, Bendl, Prashant, Hong, Casey, Alvia, Shao, Argyriou, Therrien, Venkatesh, Voloudakis, Haroutunian, Fullard, and Roussos 2026). Consider the dataset `PsychAD` storing observed cells as rows where column `PTPRG` is the number of counts for this gene, `libSize` is the total counts for each cell, `Dx` is Alzheimer's disease status, `Age` and `Sex` indicate subject age and sex, and `SubID` indicates which subject each observed cell is from. Here we use a negative binomial mixed model to test if the counts

of the PTPRG gene are associated with Alzheimer's disease status after accounting for the total number of counts for each cell. Age and sex are included as covariates and subject is a random effect to account for the repeated measures design. We use the familiar formula syntax, set `log(libSize)` to be an `offset` term, and use the `fastglmm.nb()` function:

```
data(PsychAD)
form <- PTPRG ~ offset(log(libSize)) + Dx + Age + Sex + (1|SubID)
fit <- fastglmm.nb(form, PsychAD)
fit
#>
#> Call:
#> fastglmm(formula = formula, data = data, family = negative.binomial(NA),
#>           weights = weights, maxit = maxit, tol = tol, tol.eta = tol.eta,
#>           doCoxReid = doCoxReid, nthreads = nthreads)
#>
#> Coefficients:
#> (Intercept)          DxAD           Age          SexMale
#>   -8.577681     1.102755   -0.007262   -0.103277
```

Note that while 71.2% of the 60k observations are zero, this is due to a low count rate rather than zero inflation. This is supported by comparison with the zero inflation component using `glmmTMB` (p-value of zero-inflation intercept: 0.961), and consistent with other work on single cell transcriptomics (Jiang, Sun, Song, and Li 2022; Svensson 2020; Sarkar and Stephens 2021).

The PTPRG gene has increased expression in subjects with Alzheimer's disease compared with controls with log fold change 1.10 and a p-value $< 2 \times 10^{-16}$. The count data shows high overdispersion with $\theta = 0.2443$, where $\theta \rightarrow \infty$ is equivalent to a Poisson distribution with no overdispersion.

```
summary(fit)
#> Generalized linear mixed model fit by PQL ['fastglmm']
#> Family: Negative Binomial(0.2443) ( log )
#> Formula: PTPRG ~ offset(log(libSize)) + Dx + Age + Sex + (1 | SubID)
#>
#> Coefficients:
#>              Estimate Std. Error   ddf t value Pr(>|t|)
#> (Intercept)  -8.577681   0.311172 149.9  -27.566  <2e-16 ***
#> DxAD         1.102755   0.073541 145.6   14.995  <2e-16 ***
#> Age         -0.007262   0.003775 148.2   -1.923   0.0563 .
#> SexMale     -0.103277   0.075756 145.7   -1.363   0.1749
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual df: 60227.2
#>
```

```
#> Variance components:
#>   sigSq_g: 0.4803
#>   sigSq_e: 8.17
#>   delta:  17.01
```

Variance partitioning analysis quantifies the contribution of each variable to the observed variance in PTPRG expression. Alzheimer's disease status explains 3.8% of observed expression variation, variation across subjects explains 6.0%, with sex and age making a smaller contribution. Measurement error due to sampling a finite number of counts (i.e. `CountNoise`) explains 39.4% of the variance, and the remaining 50.6% of the variance is explained by the residuals.

```
varpart(fit)
#>      Dx      Age      Sex      SubID CountNoise Residuals
#> 0.038179 0.000630 0.000332 0.060329 0.394456 0.506071
```

3. User interface

The **fastglmm** package is designed to be accessible to users already familiar with mixed model software in R. The package consists of 3 main functions for fitting mixed models (Table 1). `fastlmm()` fits a weighted LMM. `fastglmm()` fits a GLMM with 6 family functions currently supported (Table 2). `fastglmm.nb()` fits a negative binomial mixed model with the overdispersion parameter θ estimated from the data, and is equivalent to `fastglmm(..., family=negative.binomial(NA))`. The returned model fits support R generics and other key functions to extract results (Table 3).

Function	Description
<code>fastlmm()</code>	Fast linear mixed model with 1 random effect
<code>fastglmm()</code>	Generalized linear mixed model via PQL
<code>fastglmm.nb()</code>	Negative binomial mixed model via PQL

Table 1: Model fitting functions.

Family	Description
<code>gaussian()</code>	identity link
<code>binomial(link)</code>	"logit" or "probit" regression
<code>poisson()</code>	Poisson regression with log link
<code>quasibinomial(link)</code>	quasi-binomial with either "logit" or "probit" link
<code>quasipoisson()</code>	quasi-poisson with log link
<code>negative.binomial(theta)</code>	negative-binomial with log link

Table 2: Supported GLM families. For the negative binomial model, if `theta=NA` the overdispersion parameter is estimated from the data. Otherwise the given value of `theta` is used.

Function	Description
<code>anova</code>	ANOVA Tables
<code>coef</code>	Extract Model Coefficients
<code>cooks.distance</code>	Cook's Distance Metric
<code>ddf</code>	Denominator degrees of freedom using Satterthwaite method
<code>deviance</code>	Model Deviance
<code>dispersion</code>	Overdispersion parameter
<code>df.residual</code>	Residual Degrees-of-Freedom
<code>edf</code>	Effective Degrees-of-Freedom of Model Fit
<code>extractAIC</code>	Extract AIC from a Fitted Model
<code>family</code>	Extract Model Family
<code>fitted</code>	Extract Model Fitted Values
<code>fixef</code>	Extract Fixed Effects
<code>hatvalues</code>	Return Diagonals of Hat Matrix
<code>linearHypothesis</code>	Test Linear Hypothesis
<code>logLik</code>	Extract Log-Likelihood
<code>nobs</code>	Extract the Number of Observations from a Fit
<code>plot</code>	Diagnostic Plots for Model Fits
<code>predict</code>	Model Predictions
<code>ranef</code>	Extract the Modes of the Random Effects
<code>residuals</code>	Extract Model Residuals
<code>rstudent</code>	Extract Studentized Residuals
<code>sigma</code>	Extract Residual Standard Deviation
<code>summary</code>	Object Summaries and Hypothesis Tests
<code>terms</code>	Model Terms
<code>varpart</code>	Variance Partitioning Analysis
<code>vcov</code>	Calculate Variance-Covariance Matrix for Fitted Model

Table 3: **Key methods for returned model fit.**

4. Models and Methods

Here we formally describe the statistical model and efficient algorithms for parameter estimation in LMMs and GLMMs.

4.1. Linear mixed models with a single random effect

Given a vector $\mathbf{y}_{n \times 1}$ of response values for n samples, fixed effects design matrix $\mathbf{X}_{n \times c}$ for c covariates including an intercept term, coefficient vector $\boldsymbol{\beta}_{c \times 1}$, random effects vector $\boldsymbol{\gamma}_{n \times 1}$, covariance matrix $\mathbf{K}_{n \times n}$, error vector $\boldsymbol{\varepsilon}_{n \times 1}$, $n \times n$ identity matrix $\mathbf{I}$, and variance parameters σ_g^2 and σ_e^2 , the linear mixed model is

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\gamma} + \boldsymbol{\varepsilon} \quad (1)$$

$$\boldsymbol{\gamma} \sim \mathcal{N}(0, \mathbf{K}\sigma_g^2) \quad (2)$$

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \mathbf{I}\sigma_e^2), \quad (3)$$

with log-likelihood

$$\ell(\boldsymbol{\beta}, \sigma_g^2, \sigma_e^2) = \log \mathcal{N}(\mathbf{y} | \mathbf{X}\boldsymbol{\beta}, \mathbf{K}\sigma_g^2 + \mathbf{I}\sigma_e^2). \quad (4)$$

This form is often used in the genetics literature where $\mathbf{K}$ is the genetic relatedness matrix (GRM) storing the genetic relatedness between all pairs of subjects.

In the statistics literature, this model is more often formulated as

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}\boldsymbol{\alpha} + \boldsymbol{\varepsilon} \quad (5)$$

$$\boldsymbol{\alpha} \sim \mathcal{N}(0, \sigma_g^2) \quad (6)$$

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \mathbf{I}\sigma_e^2), \quad (7)$$

with $\mathbf{K} = \mathbf{Z}\mathbf{Z}^T$, where $\mathbf{Z}_{n \times q}$ is the random effect design matrix for q groups, and the random effects vector $\boldsymbol{\alpha}$ is $q \times 1$ with one entry per group. The algorithms used by the **lme4** package take advantage of the sparsity of $\mathbf{Z}$ to accelerate computation (Bates *et al.* 2015).

Alternatively, Lippert *et al.* (2011) proposed a different algorithm that takes advantage of the spectral decomposition of $\mathbf{K}$ or $\mathbf{Z}$ to transform the data so that the covariance becomes diagonal. We generalize this algorithm and apply it to LMMs and GLMMs with a single variance component. Here we give an intuitive description of the approach and describe the technical details below.

Consider estimating the parameters $\boldsymbol{\beta}$, σ_g^2 and σ_e^2 by maximizing the log-likelihood of Eqn (4). Naively, the model is expensive to fit because inverting $\mathbf{K}$ requires $\mathcal{O}(n^3)$ time. Instead, following Lippert *et al.* (2011), let the spectral decomposition of $\mathbf{K}$ be $\mathbf{U}\mathbf{S}\mathbf{U}^T$. Let $\tilde{\mathbf{y}} = \mathbf{U}^T\mathbf{y}$ and $\tilde{\mathbf{X}} = \mathbf{U}^T\mathbf{X}$ be the rotated response vector and design matrix, so that the covariance matrix of the rotated response is now diagonal. The log-likelihood of the rotated response is then

$$\ell(\boldsymbol{\beta}, \sigma_g^2, \sigma_e^2) = \log \mathcal{N}(\tilde{\mathbf{y}} | \tilde{\mathbf{X}}\boldsymbol{\beta}, \mathbf{S}\sigma_g^2 + \mathbf{I}\sigma_e^2). \quad (8)$$

By diagonalizing the covariance matrix, maximum likelihood estimates of the parameters $\beta, \sigma_g^2, \sigma_e^2$ can be obtained efficiently since the log-likelihood can now be evaluated in $\mathcal{O}(n)$.

In order to apply this to LMMs in practice we need to efficiently obtain the spectral decomposition of $\mathbf{K}$ using only $\mathbf{Z}$. To use this framework for estimation in GLMMs, we need to add sample-level weights to the LMM. Below, we describe the mathematical derivations and algorithms need to implement this in practice.

4.2. Weighted linear mixed models

Let $\mathbf{w}_{n \times 1}$ be a vector of sample-level weights. Incorporating these weights into the LMM gives the log-likelihood

$$\ell(\beta, \sigma_g^2, \sigma_e^2) = \log \mathcal{N}(\mathbf{y} | \mathbf{X}\beta, \mathbf{K}\sigma_g^2 + \mathbf{W}^{-1}\sigma_e^2), \quad (9)$$

where $\mathbf{W}$ is a, $n \times n$ diagonal matrix with entries corresponding to entries in $\mathbf{w}$. By pre-weighting $\mathbf{y}$, $\mathbf{X}$ and $\mathbf{Z}$ we can efficiently evaluate the log-likelihood as a function of the weighted data. Letting

$$\mathbf{y}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{y} \quad (10)$$

$$\mathbf{X}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{X} \quad (11)$$

$$\mathbf{Z}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{Z} \quad (12)$$

$$\mathbf{K}_w = \mathbf{Z}_w \mathbf{Z}_w^T, \quad (13)$$

we can then restate the log-likelihood as

$$\ell(\beta, \sigma_g^2, \delta) = \log \mathcal{N}(\mathbf{y}_w | \mathbf{X}_w \beta, \mathbf{K}_w \sigma_g^2 + \mathbf{I} \sigma_e^2) + \sum_i \log(\mathbf{w}_i), \quad (14)$$

following derivations in Section 11.1.

Reparameterizing by letting $\delta = \sigma_e^2 / \sigma_g^2$, the log-likelihood can be rewritten as

$$\ell(\beta, \sigma_g^2, \delta) = \log \mathcal{N}(\mathbf{y}_w | \mathbf{X}_w \beta, \sigma_g^2 (\mathbf{K}_w + \mathbf{I} \delta)) + \sum_i \log(\mathbf{w}_i) \quad (15)$$

We can then write the log-likelihood in a simpler form with the derivations shown in Section 11. Letting the spectral decomposition of $\mathbf{K}_w$ be $\mathbf{U} \mathbf{S} \mathbf{U}^T$ with eigen values stored in the vector $\mathbf{s}$, and using standard properties of linear algebra, the log determinant and quadratic form of the normal log-likelihood simplify according to

$$\log |\mathbf{K}_w + \mathbf{I} \delta| = \sum_i \log(\mathbf{s}_i + \delta) \quad (16)$$

$$(\mathbf{y}_w - \mathbf{X}_w \beta)^T (\mathbf{K}_w + \mathbf{I} \delta)^{-1} (\mathbf{y}_w - \mathbf{X}_w \beta) = \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}} \beta)_i^2}{\mathbf{s}_i + \delta} \quad (17)$$

where $\tilde{\mathbf{y}} = \mathbf{U}^T \mathbf{y}_w$ and $\tilde{\mathbf{X}} = \mathbf{U}^T \mathbf{X}_w$.

The log-likelihood in Eqn 15 then reduces to

$$\ell(\beta, \sigma_g^2, \delta) = -\frac{1}{2} \left(n \log(2\pi\sigma_g^2) + \sum_i \log(s_i + \delta) + \frac{1}{\sigma_g^2} \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta)_i^2}{s_i + \delta} \right) \quad (18)$$

Given the spectral decomposition of $\mathbf{K}_w$, the projection of $\mathbf{y}$ and $\mathbf{X}$ are now linear in n , and evaluating the log-likelihood is now linear in n . This form of the log-likelihood can now be used for parameter estimation as follows.

Regression coefficients

Find the maximum likelihood estimates of β , by taking the gradient of the log-likelihood:

$$\frac{\partial \ell(\beta, \sigma_g^2, \delta)}{\partial \beta} = \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) / \sigma_g^2. \quad (19)$$

Setting this to zero and solving for β gives

$$0 = \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) \quad (20)$$

$$= \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{y}} - \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}}\beta, \quad (21)$$

to give coefficient estimates

$$\hat{\beta} = \left(\tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}} \right)^{-1} \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{y}}. \quad (22)$$

Variance parameter σ_g^2

Given $\hat{\beta}$ and a fixed value of δ , find the maximum likelihood estimate of σ_g^2 by setting the gradient to zero according to

$$\frac{\partial \ell(\beta, \sigma_g^2, \delta)}{\partial \sigma_g^2} = -\frac{1}{2} \left(\frac{n}{\sigma_g^2} - \frac{1}{\sigma_g^4} \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta)_i^2}{s_i + \delta} \right), \quad (23)$$

to give the estimate

$$\hat{\sigma}_a^2 = \frac{1}{n} \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta)_i^2}{s_i + \delta}. \quad (24)$$

Variance ratio δ

Given closed form solutions for $\hat{\beta}$ and $\hat{\sigma}_a^2$ as a function of δ , the maximum likelihood estimates of δ can be obtained using 1 dimensional optimization like Brent's method. For a given value of δ , compute $\hat{\beta}(\delta)$ and $\hat{\sigma}_a^2(\delta)$ as function of δ using Eqns (22) and (24) and estimate δ according to

$$\hat{\delta} = \underset{\delta}{\operatorname{argmax}} \ell(\hat{\beta}(\delta), \hat{\sigma}_a^2(\delta), \delta). \quad (25)$$

While δ must be positive, the optimization is performed on the scale of $\log \delta$ so the parameter is unbounded.

Other key quantities

The variance-covariance matrix of the coefficient estimates is

$$\text{var}(\hat{\beta}) = \left(\tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}} \right)^{-1} \hat{\sigma}_a^2. \quad (26)$$

The best linear unbiased predictor (BLUP) of α is the conditional expected value given the response vector

$$\hat{\alpha} = E[\alpha | \mathbf{y}] \quad (27)$$

$$= \sqrt{\mathbf{s}} \cdot (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) / (\mathbf{s} + \delta), \quad (28)$$

as is shown in Section 11.1.

4.3. Spectral decomposition of $\mathbf{K}_w$

Naively, constructing $\mathbf{K}_w$ and computing the spectral decomposition from this $n \times n$ matrix is requires $\mathcal{O}(n^3)$ time. This can be accelerated substantially using properties of linear algebra and the spectral decomposition. Since $\mathbf{K}_w = \mathbf{U}\mathbf{S}\mathbf{U}^T$, the spectral decomposition of $\mathbf{Z}_w$ is $\mathbf{U}\mathbf{S}^{\frac{1}{2}}\mathbf{V}^T$ since

$$\mathbf{K}_w = \mathbf{Z}_w \mathbf{Z}_w^T \quad (29)$$

$$= \mathbf{U}\mathbf{S}^{\frac{1}{2}}\mathbf{V}^T\mathbf{V}\mathbf{S}^{\frac{1}{2}}\mathbf{U}^T \quad (30)$$

$$= \mathbf{U}\mathbf{S}\mathbf{U}^T, \quad (31)$$

using the fact that $\mathbf{V}$ is orthonormal.

Directly computing the singular value decomposition of $\mathbf{Z}_w$ requires $\mathcal{O}(nq \cdot \min(n, q))$ time, and when $q \ll n$ this simplifies to $\mathcal{O}(nq^2)$. In the case where the random effect comprises distinct groups as in all **lme4** models with one random effect, then $\mathbf{Z}$ is an indicator matrix where $\mathbf{Z}_{i,j}$ is 1 when sample i belongs in group j and zero otherwise. Since a sample can only belong to one group, $\mathbf{Z}$ and $\mathbf{Z}_w$ are each orthogonal. The spectral decomposition of $\mathbf{Z}_w$ can then be computed in $\mathcal{O}(nq)$ using only multiplication and taking advantage of the fact that the matrix is sparse. In this case, the resulting left singular vectors in $\mathbf{U}$ are also sparse.

Algorithm 1 describes how to compute the spectral decomposition of the weighted $\mathbf{Z}$ matrix both in the case of an indicator matrix or the general case.

4.4. Weighted linear mixed models using a low rank $\mathbf{Z}$

So far, we have assumed that $\mathbf{K}$ is full rank and considered all n singular vectors and values. Instead, let r be the number of non-zero singular values. When $\mathbf{Z}$ is an indicator matrix, as it is in all single random effect models supported by **lme4**, then $r = q$, and $q \ll n$. In the case of a general $\mathbf{Z}$ matrix, $r \leq q$. In Section 11.2 we give the derivation of the low rank case. Since only the first r singular vectors and values contribute to the model, let $\mathbf{U}_1$ be the set of left singular vectors with non-zero singular values stored in $\mathbf{s}_1$. Parameter estimates from a weighted LMM are obtained using Algorithm 2 based on this low rank method.

Algorithm 1 Spectral decomposition of weighted $\mathbf{Z}$

Require: Input value $\mathbf{Z}, \mathbf{w}$ $\mathbf{Z}_w \leftarrow \mathbf{W}^{\frac{1}{2}} \mathbf{Z}$ **if** $\mathbf{Z}$ is indicator matrix **then** $\mathbf{S} \leftarrow \mathbf{w}^T \mathbf{Z} \quad \triangleright \mathcal{O}(n)$ $\mathbf{U} \leftarrow \mathbf{Z}_w \mathbf{S}^{-\frac{1}{2}} \quad \triangleright \mathcal{O}(nq)$ $\mathbf{V} \leftarrow \mathbf{I}_p$ sort diagonal entries of $\mathbf{S}$ to be decreasing, and reorder matching columns of $\mathbf{U}$ **else** $\mathbf{U}, \mathbf{S}, \mathbf{V} \leftarrow \text{svd}(\mathbf{Z}_w) \quad \triangleright \mathcal{O}(nq \cdot \min(n, q))$ **endif****Return** $\mathbf{U}, \mathbf{S}$

Algorithm 2 `fastlmm`: Fit weighted linear mixed model with a single random effect

Require: Input value $\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{y}, \mathbf{w}$ Preprocessing $\mathbf{y} \leftarrow \mathbf{W}^{\frac{1}{2}} \mathbf{y} \quad \triangleright$ Overwrite $\mathbf{y}, \mathbf{X}$ just to simplify notation $\mathbf{X} \leftarrow \mathbf{W}^{\frac{1}{2}} \mathbf{X}$ $\tilde{\mathbf{y}} \leftarrow \mathbf{U}_1^T \mathbf{y}$ $\tilde{\mathbf{X}} \leftarrow \mathbf{U}_1^T \mathbf{X}$ $\mathbf{\Gamma}_{X,X} \leftarrow \mathbf{X}^T \mathbf{X} - \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$ $\mathbf{\Gamma}_{X,y} \leftarrow \mathbf{X}^T \mathbf{y} - \tilde{\mathbf{X}}^T \tilde{\mathbf{y}}$ Define function $\ell(\delta)$ $\triangleright$ log-likelihood function $\mathbf{D} \leftarrow \text{diag}(1/(\mathbf{s}_1 + \delta))$ $\mathbf{Q}_{X,X} \leftarrow \tilde{\mathbf{X}}^T \mathbf{D} \tilde{\mathbf{X}} + \mathbf{\Gamma}_{X,X}/\delta$ $\mathbf{Q}_{X,y} \leftarrow \tilde{\mathbf{X}}^T \mathbf{D} \tilde{\mathbf{y}} + \mathbf{\Gamma}_{X,y}/\delta$ $\hat{\boldsymbol{\beta}} \leftarrow \mathbf{Q}_{X,X}^{-1} \mathbf{Q}_{X,y}$ $\triangleright$ coefficient estimates $\mathbf{r} \leftarrow \mathbf{y} - \mathbf{X} \hat{\boldsymbol{\beta}}$ $\triangleright$ residuals $\tilde{\mathbf{r}} \leftarrow \tilde{\mathbf{y}} - \tilde{\mathbf{X}} \hat{\boldsymbol{\beta}}$ $\triangleright$ residuals in projected space $\mathbf{Q}_{r,r} \leftarrow \tilde{\mathbf{r}}^T \mathbf{D} \tilde{\mathbf{r}} + \mathbf{r}^T \mathbf{r}/\delta - \tilde{\mathbf{r}}^T \tilde{\mathbf{r}}/\delta$ $\hat{\sigma}_g^2 \leftarrow \mathbf{Q}_{r,r}/n$ **Return** $-\frac{n}{2} \log(2\pi \hat{\sigma}_g^2) - \frac{1}{2} (\sum_{i=1}^r \log(\mathbf{s}_i + \delta) + (n - k) \log \delta) - \frac{n}{2} \quad \triangleright$ log-likelihoodMLE $\hat{\delta} \leftarrow \text{argmax}_{\delta} \ell(\delta)$ $\triangleright$ Maximize log-likelihood with respect to δ $\hat{\sigma}_e^2 \leftarrow \hat{\delta} \hat{\sigma}_g^2$ $\hat{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}} \leftarrow \mathbf{Q}_{X,X}^{-1} \hat{\sigma}_g^2$ $\triangleright$ Variance-covariance matrix of $\hat{\boldsymbol{\beta}}$ $\hat{\boldsymbol{\alpha}} \leftarrow \sqrt{\mathbf{s}} \cdot \tilde{\mathbf{r}} / (\mathbf{s} + \delta)$ $\triangleright$ BLUP**Return** $\hat{\sigma}_e^2, \hat{\sigma}_g^2, \hat{\boldsymbol{\beta}}, \hat{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}, \hat{\boldsymbol{\alpha}}$

Time complexity

Consider that ordinary least squares for fitting fixed effects models requires $\mathcal{O}(nc^2 + c^3)$ time. Fitting a LMM using the naive algorithm using $\mathbf{K}$ and ignoring any sparsity is intractable since it requires $\mathcal{O}(n^3)$ time just to invert $\mathbf{K}$. Based on the work of Lippert *et al.* (2011), the algorithm used here pre-computes the SVD of $\mathbf{Z}$, projects $\mathbf{y}$ and $\mathbf{X}$ into this space, and then estimates δ by maximizing $\ell(\hat{\beta}(\delta), \hat{\sigma}_a^2(\delta), \delta)$. In the general case, parameter estimation requires time linear in n and quadratic in q , while using an indicator matrix for $\mathbf{Z}$ reduces the time to be linear in q (Table 4).

Form of $\mathbf{Z}$	SVD	Projection	Maximize $\ell(\cdot)$
General	$\mathcal{O}(nq^2)$	$\mathcal{O}(nrc)$	$\mathcal{O}((n + c^3 + nc^2)t)$
Indicator	$\mathcal{O}(nq)$	$\mathcal{O}(nc)$	$\mathcal{O}((n + c^3 + nc^2)t)$

Table 4: **Time complexity for performing each step of LMM algorithm.** Let n be the sample size, q be the number of columns in the matrix $\mathbf{Z}$ with rank r , c be the number of covariates in $\mathbf{X}$, and t be the number of times the log-likelihood is evaluated in order to estimate δ . Note that when $\mathbf{Z}$ is an indicator matrix, both $\mathbf{Z}$ and $\mathbf{U}_1$ are sparse.

4.5. Generalized linear mixed models

Just as a GLM uses iteratively reweighted least squares to solve a weight linear problem, penalized quasi-likelihood (Breslow and Clayton 1993) estimates parameters for a GLMM using an iteratively reweighted linear mixed model with updates to the weights and the working response at each iteration. Parameter estimates for GLMMs can be obtained using Algorithm 3 to iteratively call Algorithm 2. The time complexity is the same as for Algorithm 2, just scaled by the number of iterations.

Algorithm 3 fastglmm: Fit generalized linear mixed model

Require: Input value $\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{Z}, \mathbf{y}, \mathbf{w}, \text{family}$ $\triangleright$ family gives $g^{-1}(\cdot), g'(\cdot), \text{var}(\cdot)$

Initialize

$\hat{\beta} \leftarrow$ from GLM fit of $\mathbf{X}, \mathbf{y}, \mathbf{w}$

$\hat{\alpha} \leftarrow 0$

Until convergence

$\boldsymbol{\eta} \leftarrow \mathbf{X}\hat{\beta} + \mathbf{Z}\hat{\alpha}$ $\triangleright$ Linear predictor

$\boldsymbol{\mu} \leftarrow g^{-1}(\boldsymbol{\eta})$ $\triangleright$ Apply inverse link function

$\mathbf{z} \leftarrow \boldsymbol{\eta} + (\mathbf{y} - \boldsymbol{\mu})/g'(\boldsymbol{\mu})$ $\triangleright$ Working response

$\mathbf{w} \leftarrow \mathbf{w} \cdot g'(\boldsymbol{\mu})/\text{var}(\boldsymbol{\mu})$ $\triangleright$ Update weights

Evaluate **fastlmm**($\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{z}, \mathbf{w}$)

Return $\hat{\sigma}_e^2, \hat{\sigma}_g^2, \hat{\beta}, \hat{\Sigma}_\beta, \hat{\alpha}, \boldsymbol{\mu}$

4.6. Negative binomial mixed models

In addition to fixed and random effects parameters, fitting a negative binomial mixed model (NBMM) requires estimating the overdispersion parameter θ . Parameter estimation is performed by alternately fitting a NBMM with a fixed value of θ , and then updating the value of θ based on this fit. Parameter estimates for NBMM can be obtained using Algorithm 4 to iteratively call Algorithm 3. The time complexity is the same as for Algorithm 3, just scaled by the number of iterations.

Algorithm 4 `fastglmm.nb`: Fit negative binomial mixed model

Require: Input value $\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{Z}, \mathbf{y}, \mathbf{w}$

 Initialize

 Evaluate `fastglmm`($\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{Z}, \mathbf{y}, \mathbf{w}, \text{poisson}$)

$\triangleright$ Fit Poisson GLMM

$\hat{\theta} \leftarrow \text{nb_theta}(\mathbf{y}, \boldsymbol{\mu})$

$\triangleright$ Estimate θ from model fit

Until convergence

 Evaluate `fastglmm`($\mathbf{U}_1, \mathbf{s}_1, \mathbf{X}, \mathbf{Z}, \mathbf{y}, \mathbf{w}, \text{NB}(\hat{\theta})$)

$\triangleright$ Fit NBMM with current $\hat{\theta}$

$\hat{\theta} \leftarrow \text{nb_theta}(\mathbf{y}, \boldsymbol{\mu})$

Return $\hat{\sigma}_e^2, \hat{\sigma}_g^2, \hat{\boldsymbol{\beta}}, \hat{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}, \hat{\boldsymbol{\alpha}}, \hat{\theta}$

This uses the `nb_theta()` function in the `fastglmm` package to estimate θ . Given observed response $\mathbf{y}$ and the conditional mean $\boldsymbol{\mu}$, the log-likelihood of θ is

$$\begin{aligned} \ell(\theta|\mathbf{y}, \boldsymbol{\mu}) &= \sum_{i=1}^n [\log \Gamma(y_i + \theta) - \log \Gamma(\theta) - \log(y_i!) + \theta \log \theta - (y_i + \theta) \log(\theta + \mu_i) + y_i \log \mu_i] \\ &\propto -n \log \Gamma(\theta) + n\theta \log \theta + \sum_{i=1}^n [\log \Gamma(y_i + \theta) - (y_i + \theta) \log(\theta + \mu_i)], \end{aligned} \quad (33)$$

which can be evaluated in $\mathcal{O}(n)$ time. Evaluating the term $\log \Gamma(y_i + \theta)$ for all samples is computationally expensive, but can be accelerated by only evaluating it for unique values of y_i and scaling by the multiplicity (Ahlmann-Eltze and Huber 2020).

The maximum likelihood estimate is obtained by maximizing Eqn (33) with respect to θ using Brent's method (1973) for univariate optimization. While θ must be positive, the optimization is performed on the scale of $\log \theta$ so the parameter is unbounded. This approach is fast, but has the limitation that it does not give the standard error of $\hat{\theta}$ since the Hessian of the log-likelihood is not evaluated.

5. Summarizing the model fit

Here, we consider summarizing the model fit using hypothesis testing and variance partitioning analysis.

5.1. Hypothesis testing

Univariate testing of the null model $H_0 : \hat{\beta}_i = 0$ is performed with a Wald test using the

statistic

$$t = \frac{\mathbf{L}\boldsymbol{\beta}}{\sqrt{\mathbf{L}\hat{\boldsymbol{\Sigma}}_{\boldsymbol{\beta}}\mathbf{L}^T}}, \quad (34)$$

which is asymptotically normally distributed under H_0 according to

$$t \sim \mathcal{N}(0, 1). \quad (35)$$

Yet with small sample size, this asymptotic null distribution can be anti-conservative and give an elevated false positive rate. Instead, we use a finite-sample Student's t approximation to the null distribution using the Satterthwaite method to approximate the degrees of freedom, ν (Kuznetsova, Brockhoff, and Christensen 2017; Christensen 2020). This approach to estimate ν from the data has been successfully applied by **lmerTest**, **mmrm** and **glmmTMB** for LMMs, and further extended to GLMMs by **glmmTMB**. While this can be computationally demanding in the general case, we take advantage of the special case of a single random effect to estimate ν with negligible additional cost (Section 11.3).

Given the estimate of ν , the statistic t is distributed according to

$$t \stackrel{\text{approx}}{\sim} t_{\nu}. \quad (36)$$

5.2. Residual degrees of freedom

Letting $\mathbf{H}$ be the hat matrix mapping from observed to predicted responses so that

$$\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}, \quad (37)$$

then the degrees of freedom of the model fit is $\text{tr}(\mathbf{H})$ (Hastie and Tibshirani 1990; Ye 1998). In the case of the linear model with p predictors, then $\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ and $\text{tr}(\mathbf{H}) = p$.

The residual degrees of freedom, ν_{rdf} is then

$$\nu_{rdf} = \text{tr}[(\mathbf{I} - \mathbf{H})^T(\mathbf{I} - \mathbf{H})] \quad (38)$$

$$= n - 2\text{tr}(\mathbf{H}) + \text{tr}(\mathbf{H}\mathbf{H}^T). \quad (39)$$

In the case of the linear model, $\nu_{rdf} = n - p$. In general, computing $\text{tr}(\mathbf{H}\mathbf{H}^T)$ is expensive, but can be approximated by $\text{tr}(\mathbf{H})$ (Hastie and Tibshirani 1990). In that case, the approximate degrees of freedom is

$$\hat{\nu}_{rdf} = n - \text{tr}(\mathbf{H}). \quad (40)$$

This is exact in the case of a linear model, and approximate for LMMs. Importantly, this approximation is fast since it depends only on the diagonals of the hat matrix.

5.3. Variance Partitioning analysis

In a LMM, the total variance in the response can be partitioned into components for the fixed effects, random effects and residual variance. Here, we quantify the contribution of each component to the total variance in the response.

Let $\boldsymbol{\eta} = \mathbf{X}\hat{\boldsymbol{\beta}}$ be the systematic part of the model capturing the signal from the fixed effects, and let $\sigma_{\text{signal}}^2 = \text{var}(\boldsymbol{\eta})$ be the total variance of the fixed components. The variance explained by the random effect is σ_g^2 and let σ_{unexpl}^2 be the unexplained variance not captured by the signal components. Let $\mathbf{X}_j\hat{\boldsymbol{\beta}}_j$ be the linear predictor of the j^{th} component, where the subscript j indicates the columns of the design matrix and coefficient vector corresponding to the j^{th} fixed effect. Letting $\sigma_j^2 = \text{var}(\mathbf{X}_j\hat{\boldsymbol{\beta}}_j)$ be the variance of the j^{th} component, then $\gamma_j = \sigma_j^2 / \sum_k \sigma_k^2$ is the fraction of the signal variance explained by fixed effect j . This accounts for correlation between the fixed effects components.

We can then quantify the contribution of each component of variance. The fraction of variance explained by the j^{th} fixed effect component is

$$\frac{\gamma_j \sigma_j^2}{\sigma_{\text{signal}}^2 + \sigma_g^2 + \sigma_{\text{unexpl}}^2}, \quad (41)$$

the fraction explained by the random effect component is

$$\frac{\sigma_g^2}{\sigma_{\text{signal}}^2 + \sigma_g^2 + \sigma_{\text{unexpl}}^2}, \quad (42)$$

and fraction of unexplained variance is

$$\frac{\sigma_{\text{unexpl}}^2}{\sigma_{\text{signal}}^2 + \sigma_g^2 + \sigma_{\text{unexpl}}^2}. \quad (43)$$

In this case, the unexplained component is the residual variance. We have used this formulation of partitioning variance in LMMs in previous work ([Hoffman and Schadt 2016](#)).

In the case of a GLMM, the signal component is the same but the unexplained variance is different because the linear predictor $\boldsymbol{\eta}$ acts on the latent scale. All variances are expressed on the latent scale, and the unexplained variance depends on the distribution and link function of model ([Nakagawa, Johnson, and Schielzeth 2017](#); [Nakagawa and Schielzeth 2013](#)). In the Gaussian case with identity link, the unexplained variance is simply the residual variance, but for other models this term is more complicated (Table 5). For count models, we used exact and approximate methods described by [Hoffman and Roussos \(2026\)](#).

Family	Link	Unexplained variance	Approximation
Gaussian	identity	$\hat{\sigma}_e^2$	
Binomial	logit	$\pi^2/3$	
Binomial	probit	1	
Poisson	log	no closed form	$1/\hat{\lambda}$
Quasi-poisson	log	no closed form	$\hat{\omega}/\hat{\lambda}$
NB	log	no closed form	$1/\hat{\lambda} + 1/\hat{\theta}$

Table 5: **Unexplained variance term and approximations.** Exact and approximate formulas given by [Nakagawa and Schielzeth \(2013\)](#) and [Nakagawa et al. \(2017\)](#). For count models, see [Hoffman and Roussos \(2026\)](#).

6. Implementation

The core functionality of the **fastglmm** package is written in C++17 with linear algebra implemented in the **armadillo** library (Sanderson and Curtin 2016). The C++ code is written as a header-only library so other package developers can call **fastglmm** capabilities at either the R or C++ levels. C++ templates are used for $\mathbf{U}_1$ to handle dense matrices with `arma::mat` and sparse matrices with `arma::sp_mat`. A thin layer linking the R objects to **armadillo** C++ code is written using **RcppArmadillo** (Eddelbuettel and Sanderson 2014). A thin R wrapper provides an interface for users with standard R syntax and generics.

7. Results

In order to evaluate the difference in run times across algorithms and implementations, we fit the NBMM on the **PsychAD** dataset of 60k cells from Section 2.1. **fastglmm** was the fastest method, taking 0.14 seconds per model fit, and was 12x faster than the closest competitor, **glmm.nb** (Figure 1). **GLMMadaptive**, **glmmTMB**, and **glmer.nb**(, . . . `nAGQ=0`) took between 4.7 and 13.4 seconds, while running **glmer.nb** with default parameters took 102.9 seconds. When fitting NBMMs across many genes with even larger sample sizes, this savings is substantial.

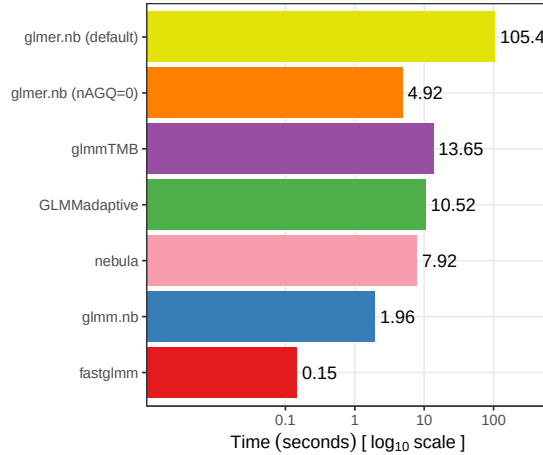


Figure 1: **Run time for PsychAD example.** Results are averaged over 100 runs. Models were fit with **fastglmm** v0.4.0, **NBZIMM** v1.0, **nebula** v1.5.6, **GLMMadaptive** v0.9.7, **glmmTMB** v1.1.14, **lme4** v2.0.1.

We also performed empirical simulations on datasets ranging from 100 to 10,000 samples to evaluate parameter estimates from each method (Figure 2). With large sample size, **fastglmm** and **glmm.nb** produce accurate parameter estimates, although they are biased at small sample sizes especially for the overdispersion parameter θ . With sufficient sample size, **fastglmm** produces estimates that are indistinguishable from the widely used **glmmTMB** and **glmer.nb** methods, while being ~ 100 - 1000 x faster. These results were robust across a range of parameter values.

In addition, the optimization approach used by **fastglmm** is very stable and nearly always converges in practice. Across a range of empirical simulations, **GLMMadaptive** and **glmm.nb**

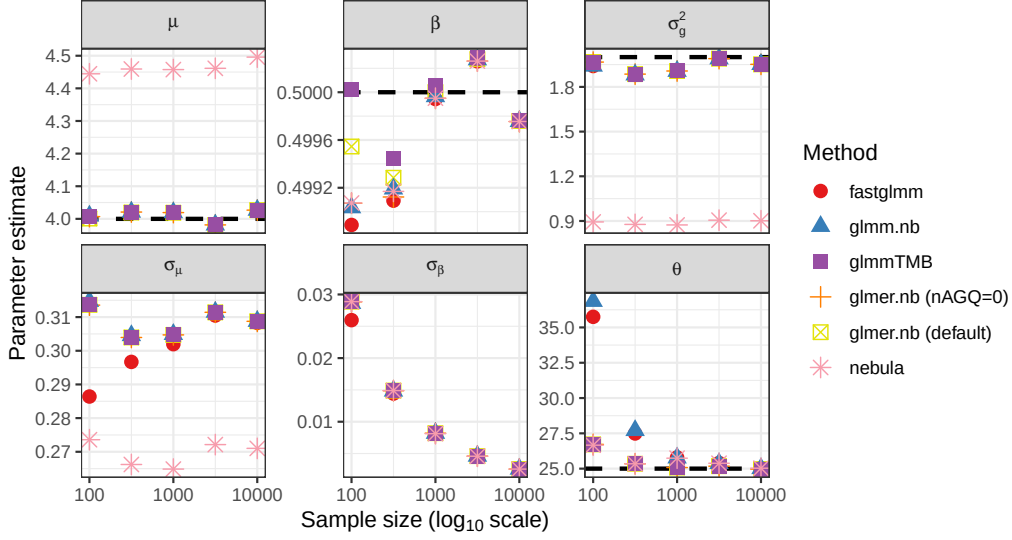


Figure 2: **Performance in empirical simulations.** Estimates are shown for each parameter and method for an increasing number of samples. Points indicate mean estimate across 100 simulations. Dashed line indicates the true parameter value, but note there is no true value for the standard errors of μ and β . Sample size ranging from 100 to 10,000 with an intercept $\mu = 4$, coefficient $\beta = 0.5$ applied to a fixed effect variable with values drawn from a standard normal. Samples were drawn from $q = 20$ groups with coefficients drawn from a normal with mean 0 and variance $\sigma_g^2 = 2$, and group was modeled as a random effect. The negative binomial overdispersion parameter was set to $\theta = 25$. GLMMadaptive produced very poor estimates and was excluded from the plot.

were both unstable, had convergence issues, and sometimes gave parameter estimates that were far from the true values. Although estimates were comparable to other methods in many scenarios, it was difficult to predict when these methods would fail.

We evaluated the calibration of small sample p-values from hypothesis testing using the asymptotic null distribution compared to the Satterthwaite method. Simulating data from Gaussian, Poisson and negative binomial models from just 12 subjects under the null model of no association between fixed effect and response, the asymptotic method gives an excess of small p-values while the Satterthwaite method gives well calibrated p-values (Figure 3). Due to the speed and correct calibration even with small sample size, the Satterthwaite method is the default for hypothesis testing in the **fastglmm** package.

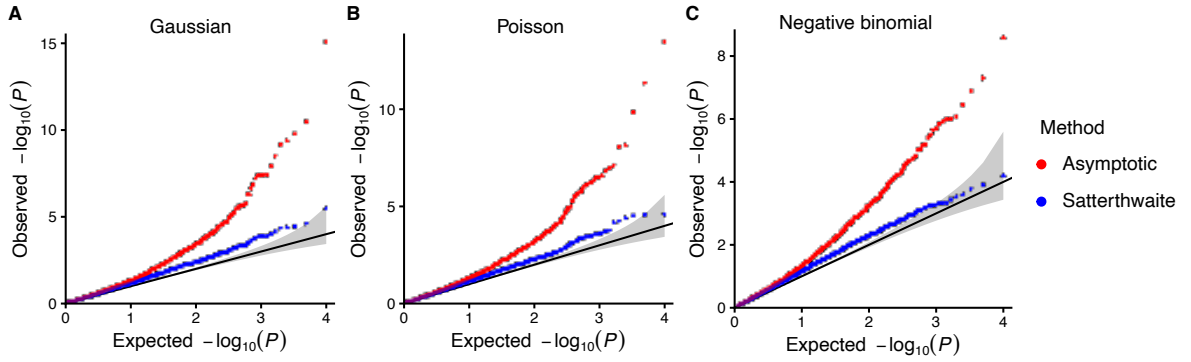


Figure 3: Calibration of p-values from hypothesis testing shown in $-\log_{10}$ quantile-quantile plot. Data was simulated for 12 subjects with 20 replicates per subject, and random effect variance set to $\sigma_g^2 = 1$. For each subject, a variable $\mathbf{x}$ was sampled from a standard normal. A model $\mathbf{y} \sim \mathbf{x} + (1|\text{Subject})$ was fit for 10,000 simulations with true coefficient for $\mathbf{x}$ equal to zero, and hypothesis testing performed on this coefficient. P-values were reported using the asymptotic null distribution (red points) and the finite sample Student's t approximation using the Satterthwaite method (blue points). Observed p-values from these methods were compared to the expected uniform distribution and shown on a $-\log_{10}$ scale. Black line indicates uniform distribution and grey region indicates 95% confidence intervals. Results are shown for 3 response types: **A)** Simulate response from Gaussian distribution with $\sigma_e^2 = 1$, **B)** Simulate response from Poisson distribution, **C)** Simulate response from negative binomial distribution with $\theta = 10$. Results are robust to changes in parameter values and sample size, with the difference between the two methods decreasing for large sample sizes.

8. Conclusion

Here, we have presented the **fastglmm** package for efficiently fitting LMMs and GLMMs on massive datasets. The methods and software we have developed gives parameter estimates that are concordant with standard packages while being orders of magnitudes faster on massive datasets of practical interest. This work aims to open up mixed models to large-scale datasets where current algorithms and implementations are intractable. Use of familiar mixed model syntax and R generics to extract results from model fits will enable users to easily integrate **fastglmm** into R workflows.

9. Acknowledgments

This work was supported by U24AG087563 from the National Institute of Aging.

10. References

- Ahlmann-Eltze C, Huber W (2020). “glmGamPoi: Fitting Gamma-Poisson generalized linear models on single cell count data.” *Bioinformatics*, **36**(24), 5701–5702. ISSN 14602059. doi:10.1093/bioinformatics/btaa1009.
- Bates D, Mächler M, Bolker B, Walker S (2015). “Fitting Linear Mixed-Effects Models Using lme4.” *Journal of Statistical Software*, **67**(1), 1–48. ISSN 1548-7660. doi:10.18637/jss.v067.i01. URL <http://www.jstatsoft.org/v67/i01/>.
- Belenky G, Wesensten NJ, Thorne DR, Thomas ML, Sing HC, Redmond DP, Russo MB, Balkin TJ (2003). “Patterns of performance degradation and restoration during sleep restriction and subsequent recovery: A sleep dose-response study.” *Journal of Sleep Research*, **12**(1), 1–12. ISSN 09621105. doi:10.1046/j.1365-2869.2003.00337.x.
- Brent R (1973). *Algorithms for Minimization Without Derivatives*. Prentice Hall.
- Breslow NE, Clayton DG (1993). “Approximate Inference in Generalized Linear Mixed Models.” *Journal of the American Statistical Association*, **88**(421), 9. ISSN 01621459. doi:10.2307/2290687. URL <https://www.jstor.org/stable/2290687?origin=crossref>.
- Christensen RHB (2020). “Satterthwaite’s Method for Degrees of Freedom in Linear Mixed Models.” pp. 1–10. URL https://github.com/runehaubo/lmerTestR/blob/master/pkg_notes/Satterthwaite_for_LMMs.html.
- Eddelbuettel D, Sanderson C (2014). “RcppArmadillo: Accelerating R with high-performance C++ linear algebra.” *Computational Statistics and Data Analysis*, **71**, 1054–1063. ISSN 01679473. doi:10.1016/j.csda.2013.02.005. URL <http://dx.doi.org/10.1016/j.csda.2013.02.005><https://linkinghub.elsevier.com/retrieve/pii/S0167947313000492>.
- Fai AHT, Cornelius PL (1996). “Approximate f-tests of multiple degree of freedom hypotheses in generalized least squares analyses of unbalanced split-plot experiments.” *Journal of Statistical Computation and Simulation*, **54**(4), 363–378. ISSN 00949655. doi:10.1080/00949659608811740.
- Hastie T, Tibshirani RJ (1990). *Generalized Additive Models*. Chapman and Hall/CRC. ISBN 9780412343902.
- He L, Davila-Velderrain J, Sumida TS, Hafler DA, Kellis M, Kulminski AM (2021). “NEBULA is a fast negative binomial mixed model for differential or co-expression analysis of large-scale multi-subject single-cell data.” *Communications Biology*, **4**(1). ISSN 23993642. doi:10.1038/s42003-021-02146-6.
- Hoffman GE, Lee D, Bendl J, Prashant N, Hong A, Casey C, Alvia M, Shao Z, Argyriou S, Therrien K, Venkatesh S, Voloudakis G, Haroutunian V, Fullard JF, Roussos P (2026).

- “Efficient differential expression analysis of large-scale single cell transcriptomics data using dreamlet.” *Nature Communications*.
- Hoffman GE, Roussos P (2025). “Fast Probabilistic Whitening Transformation for Ultra-High Dimensional Genetic Data.” doi:10.1101/2025.09.01.673591. URL <http://biorxiv.org/lookup/doi/10.1101/2025.09.01.673591>.
- Hoffman GE, Roussos P (2026). “Variance partitioning analysis for count models quantifies measurement error in single cell data.” *in prep*.
- Hoffman GE, Schadt EE (2016). “variancePartition: interpreting drivers of variation in complex gene expression studies.” *BMC Bioinformatics*, **17**(1), 483. ISSN 1471-2105. doi:10.1186/s12859-016-1323-z.
- Jiang R, Sun T, Song D, Li JJ (2022). “Statistics or biology: the zero-inflation controversy about scRNA-seq data.” *Genome Biology*, **23**(1), 1–24. ISSN 1474760X. doi:10.1186/s13059-022-02601-5.
- Kang HM, Sul JH, Service SK, Zaitlen Na, Kong SY, Freimer NB, Sabatti C, Eskin E (2010). “Variance component model to account for sample structure in genome-wide association studies.” *Nature Genetics*, **42**(4), 348–354. ISSN 1061-4036. doi:10.1038/ng.548. URL <http://www.ncbi.nlm.nih.gov/pubmed/20208533><http://www.nature.com/doifinder/10.1038/ng.548>.
- Krieger DSB, Li L, Dedic J, Kelkhoff D, Kunzmann K, Lang BM, Stock C, Wang Y, James D, Sidi J, Leibovitz D, Sjoberg DD, Ivan N (2026). “mmrm: Mixed Models for Repeated Measures.” URL <https://openpharma.github.io/mmrm/>.
- Kuznetsova A, Brockhoff PB, Christensen RHB (2017). “lmerTest Package: Tests in Linear Mixed Effects Models.” *Journal of Statistical Software*, **82**(13). ISSN 1548-7660. doi:10.18637/jss.v082.i13. URL <http://www.jstatsoft.org/v82/i13/>.
- Lippert C, Listgarten J, Liu Y, Kadie CM, Davidson RI, Heckerman D (2011). “FaST linear mixed models for genome-wide association studies.” *Nature Methods*, **8**(10), 833–835. ISSN 1548-7091. doi:10.1038/nmeth.1681. URL <http://www.ncbi.nlm.nih.gov/pubmed/21892150><https://www.nature.com/articles/nmeth.1681>.
- McGillcuddy M, Popovic G, Bolker BM, Warton DI (2025). “Parsimoniously Fitting Large Multivariate Random Effects in glmmTMB.” *Journal of Statistical Software*, **112**(1). ISSN 15487660. doi:10.18637/jss.v112.i01. 2411.04411.
- Nakagawa S, Johnson PCD, Schielzeth H (2017). “The coefficient of determination R^2 and intra-class correlation coefficient from generalized linear mixed-effects models revisited and expanded.” *Journal of The Royal Society Interface*, **14**(134), 20170213. ISSN 1742-5689. doi:10.1098/rsif.2017.0213.
- Nakagawa S, Schielzeth H (2013). “A general and simple method for obtaining R^2 from generalized linear mixed-effects models.” *Methods in Ecology and Evolution*, **4**(2), 133–142. ISSN 2041210X. doi:10.1111/j.2041-210x.2012.00261.x.
- Pinheiro J, Bates D (2000). *Mixed-effects models in S and S-Plus*. Springer, New York. doi:10.1007/978-1-4419-0318-1. URL <http://dx.doi.org/10.1007/978-1-4419-0318-1>.

- Rizopoulos D (2025). “GLMMadaptive: Generalized Linear Mixed Models using Adaptive Gaussian Quadrature.” URL <https://cran.r-project.org/package=GLMMadaptive>.
- Rood JE, Wynne S, Robson L, Hupalowska A, Randell J, Teichmann SA, Regev A (2025). “The Human Cell Atlas from a cell census to a unified foundation model.” *Nature*, **637**(8048), 1065–1071. ISSN 14764687. doi:10.1038/s41586-024-08338-4. URL <http://dx.doi.org/10.1038/s41586-024-08338-4>.
- Sanderson C, Curtin R (2016). “Armadillo: a template-based C++ library for linear algebra.” *The Journal of Open Source Software*, **1**(2), 26. doi:10.21105/joss.00026.
- Sarkar A, Stephens M (2021). “Separating measurement and expression models clarifies confusion in single-cell RNA sequencing analysis.” *Nature Genetics*, **53**(6), 770–777. ISSN 15461718. doi:10.1038/s41588-021-00873-4. URL <http://dx.doi.org/10.1038/s41588-021-00873-4>.
- Svensson V (2020). “Droplet scRNA-seq is not zero-inflated.” *Nature Biotechnology*, **38**(2), 147–150. ISSN 1087-0156. doi:10.1038/s41587-019-0379-5. URL <https://www.nature.com/articles/s41587-019-0379-5>.
- Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. Statistics and Computing. Springer New York, New York, NY. ISBN 978-1-4419-3008-8. doi:10.1007/978-0-387-21706-2. URL <http://link.springer.com/10.1007/978-0-387-21706-2>.
- Ye J (1998). “On measuring and correcting the effects of data mining and model selection.” *Journal of the American Statistical Association*, **93**(441), 120–131. URL <http://www.tandfonline.com/doi/abs/10.1080/01621459.1998.10474094>.
- Zhang X, Yi N (2020). “NBZIMM: negative binomial and zero-inflated mixed models, with application to microbiome/metagenomics data analysis.” *BMC Bioinformatics*, **21**(1), 1–19. ISSN 14712105. doi:10.1186/s12859-020-03803-z. URL <https://doi.org/10.1186/s12859-020-03803-z>.
- Zhou W, Zhao Z, Nielsen JB, Fritsche LG, LeFaive J, Gagliano Taliun SA, Bi W, Gabrielsen ME, Daly MJ, Neale BM, Hveem K, Abecasis GR, Willer CJ, Lee S (2020). “Scalable generalized linear mixed model for region-based association tests in large biobanks and cohorts.” *Nature Genetics*, **52**(6), 634–639. ISSN 1061-4036. doi:10.1038/s41588-020-0621-6. URL <http://dx.doi.org/10.1038/s41588-020-0621-6><https://www.nature.com/articles/s41588-020-0621-6>.
- Zhou X, Stephens M (2012). “Genome-wide efficient mixed-model analysis for association studies.” *Nature Genetics*, **44**(7), 821–4. ISSN 1546-1718. doi:10.1038/ng.2310. URL <http://www.nature.com/ng/journal/v44/n7/abs/ng.2310.html><http://www.nature.com/doifinder/10.1038/ng.2310><http://www.ncbi.nlm.nih.gov/pubmed/22706312>.

11. Derivations

Let $\mathbf{a} \cdot \mathbf{b}$ indicate element-wise product of vectors, and $\mathbf{a}/\mathbf{b}$ indicate element-wise division.

11.1. Weighted linear mixed models with a single random effect

Consider the weight linear mixed model with log-likelihood

$$\ell(\boldsymbol{\beta}, \sigma_g^2, \sigma_e^2) = \log \mathcal{N}(\mathbf{y} | \mathbf{X}\boldsymbol{\beta}, \mathbf{K}\sigma_g^2 + \mathbf{W}^{-1}\sigma_e^2) \quad (44)$$

Evaluating this log-likelihood naively involves inverting $\mathbf{K}\sigma_g^2 + \mathbf{W}^{-1}\sigma_e^2$ for each parameter value. Here, we develop an approach to efficiently evaluate this log-likelihood for many parameter values.

Letting $\delta = \sigma_e^2/\sigma_g^2$, the covariance term can be restated as $\sigma_g^2(\mathbf{K} + \mathbf{W}^{-1}\delta)$. The log-likelihood is then

$$\begin{aligned} \ell(\boldsymbol{\beta}, \sigma_g^2, \delta) &= \log \mathcal{N}(\mathbf{y} | \mathbf{X}\boldsymbol{\beta}, \sigma_g^2(\mathbf{K} + \mathbf{W}^{-1}\delta)) \\ &= -\frac{1}{2} \left(n \log(2\pi\sigma_g^2) + \log |\mathbf{K} + \mathbf{W}^{-1}\delta| + (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{K} + \mathbf{W}^{-1}\delta)^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) / \sigma_g^2 \right) \end{aligned} \quad (45)$$

Rewriting the covariance matrix as

$$\mathbf{K} + \mathbf{W}^{-1}\delta = \mathbf{W}^{-\frac{1}{2}} \left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + \mathbf{I}\delta \right) \mathbf{W}^{-\frac{1}{2}}, \quad (46)$$

the log determinant can be simplified as

$$\log |\mathbf{K} + \mathbf{W}^{-1}\delta| = \log \left| \mathbf{W}^{-\frac{1}{2}} \left(\mathbf{W}^{\frac{1}{2}} \mathbf{K} \mathbf{W}^{\frac{1}{2}} + \mathbf{I}\delta \right) \mathbf{W}^{-\frac{1}{2}} \right| \quad (47)$$

$$= \log |\mathbf{K}_w + \mathbf{I}\delta| + \sum_i \log(\mathbf{w}_i), \quad (48)$$

and the quadratic form can be simplified as

$$(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{K} + \mathbf{W}\delta)^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T \mathbf{W}^{\frac{1}{2}} (\mathbf{K}_w + \mathbf{I}\delta)^{-1} \mathbf{W}^{\frac{1}{2}} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \quad (49)$$

$$= (\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta})^T (\mathbf{K}_w + \mathbf{I}\delta)^{-1} (\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta}), \quad (50)$$

where

$$\mathbf{y}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{y} \quad (51)$$

$$\mathbf{X}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{X} \quad (52)$$

$$\mathbf{Z}_w = \mathbf{W}^{\frac{1}{2}} \mathbf{Z} \quad (53)$$

$$\mathbf{K}_w = \mathbf{Z}_w \mathbf{Z}_w^T \quad (54)$$

We can then restate the log-likelihood using weighted data as

$$\ell(\boldsymbol{\beta}, \sigma_g^2, \delta) = \log \mathcal{N}(\mathbf{y}_w | \mathbf{X}_w\boldsymbol{\beta}, \sigma_g^2(\mathbf{K}_w + \mathbf{I}\delta)) + \sum_i \log(\mathbf{w}_i) \quad (55)$$

Consider the spectral decomposition of $\mathbf{Z}_w = \mathbf{U}\mathbf{S}^{\frac{1}{2}}\mathbf{V}^T$ so that the spectral decomposition of $\mathbf{K}_w$ is $\mathbf{U}\mathbf{S}\mathbf{U}^T$. Using properties of linear algebra to simplify the log determinant from the log-likelihood in Eqn (55) gives

$$\log |\mathbf{K}_w + \mathbf{I}\delta| = \log |\mathbf{U}\mathbf{S}\mathbf{U}^T + \mathbf{I}\delta| \quad (56)$$

$$= \log |\mathbf{U}\mathbf{S}\mathbf{U}^T + \mathbf{U}\mathbf{U}^T\delta| \quad (57)$$

$$= \log |\mathbf{U}(\mathbf{S} + \mathbf{I}\delta)\mathbf{U}^T| \quad (58)$$

$$= \log |\mathbf{S} + \mathbf{I}\delta| \quad (59)$$

$$= \sum_i \log(s_i + \delta), \quad (60)$$

where s_i stores the i^{th} eigen value of $\mathbf{K}_w$. Similarly, the quadratic form simplifies according to

$$(\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta})^T(\mathbf{K}_w + \mathbf{I}\delta)^{-1}(\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta}) = (\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta})^T(\mathbf{U}\mathbf{S}\mathbf{U}^T + \mathbf{U}\mathbf{U}^T\delta)^{-1}(\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta}) \quad (61)$$

$$= (\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta})^T \left(\mathbf{U}(\mathbf{S} + \mathbf{I}\delta)\mathbf{U}^T \right)^{-1} (\mathbf{y}_w - \mathbf{X}_w\boldsymbol{\beta}) \quad (62)$$

$$= (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T \mathbf{U}(\mathbf{S} + \mathbf{I}\delta)^{-1} \mathbf{U}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \quad (63)$$

$$= (\mathbf{U}^T \mathbf{y}_w - \mathbf{U}^T \mathbf{X}_w\boldsymbol{\beta})^T (\mathbf{S} + \mathbf{I}\delta)^{-1} (\mathbf{U}^T \mathbf{y}_w - \mathbf{U}^T \mathbf{X}_w\boldsymbol{\beta}) \quad (64)$$

$$= (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\boldsymbol{\beta})^T (\mathbf{S} + \mathbf{I}\delta)^{-1} (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\boldsymbol{\beta}) \quad (64)$$

$$= \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\boldsymbol{\beta})_i^2}{s_i + \delta} \quad (65)$$

where $\tilde{\mathbf{y}} = \mathbf{U}^T \mathbf{y}_w$ and $\tilde{\mathbf{X}} = \mathbf{U}^T \mathbf{X}_w$.

Plugging in the simplified log determinant and quadratic form, the log-likelihood in Eqn (55) reduces to

$$\ell(\boldsymbol{\beta}, \sigma_g^2, \delta) = -\frac{1}{2} \left(n \log(2\pi\sigma_g^2) + \sum_i \log(s_i + \delta) + \frac{1}{\sigma_g^2} \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\boldsymbol{\beta})_i^2}{s_i + \delta} \right) \quad (66)$$

Given the spectral decomposition of $\mathbf{K}_w$, the projection of $\mathbf{y}$ and $\mathbf{X}$ are now linear in n , and evaluating the log-likelihood for any value of δ is now linear in n .

Parameters are estimated from the log-likelihood as described in the main text. The regression coefficients and σ_g^2 are estimated in closed form using

$$\hat{\boldsymbol{\beta}} = \left(\tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}} \right)^{-1} \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{y}} \quad (67)$$

$$\hat{\sigma}_a^2 = \frac{1}{n} \sum_i \frac{(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\boldsymbol{\beta})_i^2}{s_i + \delta} \quad (68)$$

These closed form solutions are used to estimate δ by maximizing the log-likelihood with respect to δ using a 1 dimensional search as described in the main text.

Evaluating other key quantities

Letting $\mathbf{V} = \mathbf{K}\sigma_g^2 + \mathbf{W}^{-1}\sigma_e^2$ be the covariance matrix of the untransformed data, generalized

least squares gives the variance-covariance matrix of the coefficient estimates as

$$\text{var}(\hat{\beta}) = (\mathbf{X}^T \mathbf{V}^{-1} \mathbf{X})^{-1} \hat{\sigma}_e^2. \quad (69)$$

Simplifying terms as in Eqns (61) - (65) gives

$$\text{var}(\hat{\beta}) = (\tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}})^{-1} \hat{\sigma}_a^2. \quad (70)$$

The best linear unbiased predictor (BLUP) of α is the conditional expected value given the response vector:

$$\hat{\alpha} = E[\alpha | \mathbf{y}] \quad (71)$$

$$= \mathbf{W}^{-\frac{1}{2}} E[\alpha | \mathbf{y}_w] \quad (72)$$

$$= \text{cov}(\alpha, \mathbf{y}) \text{var}(\mathbf{y})^{-1} (\mathbf{y} - \mathbf{X}\beta) \quad (73)$$

$$= \sigma_g^2 \mathbf{Z}^T (\sigma_g^2 (\mathbf{K} + \mathbf{I}\delta))^{-1} (\mathbf{y} - \mathbf{X}\beta) \quad (74)$$

$$= \mathbf{Z}^T (\mathbf{K} + \mathbf{I}\delta)^{-1} (\mathbf{y} - \mathbf{X}\beta) \quad (75)$$

$$= \mathbf{Z}^T (\mathbf{U}\mathbf{S}\mathbf{U}^T + \mathbf{U}\mathbf{U}^T\delta)^{-1} (\mathbf{y} - \mathbf{X}\beta) \quad (76)$$

$$= \mathbf{Z}^T \mathbf{U} (\mathbf{S} + \mathbf{I}\delta)^{-1} (\mathbf{U}^T \mathbf{y} - \mathbf{U}^T \mathbf{X}\beta) \quad (77)$$

$$= \mathbf{S}^{-\frac{1}{2}} \mathbf{U}^T \mathbf{U} (\mathbf{S} + \mathbf{I}\delta)^{-1} (\mathbf{U}^T \mathbf{y} - \mathbf{U}^T \mathbf{X}\beta) \quad (78)$$

$$= \mathbf{S}^{-\frac{1}{2}} (\mathbf{S} + \mathbf{I}\delta)^{-1} (\mathbf{U}^T \mathbf{y} - \mathbf{U}^T \mathbf{X}\beta) \quad (79)$$

$$= \sqrt{\mathbf{s}} \cdot (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) / (\mathbf{s} + \delta) \quad (80)$$

11.2. Low rank $\mathbf{Z}$

So far we have assumed that $\mathbf{K} = \mathbf{Z}\mathbf{Z}^T$ is full rank. In practice, $\mathbf{K}$ is often low rank so that many of its eigen values are zero. Therefore, the spectral decomposition can be partitioned into components with non-zero and zero eigen values where $\mathbf{U} = [\mathbf{U}_1 \mathbf{U}_2]$ and $\mathbf{S} = \text{diag}[\mathbf{s}_1 \mathbf{0}]$. The spectral decomposition of $\mathbf{K}$ is then

$$\mathbf{K} = \mathbf{U}\mathbf{S}\mathbf{U}^T \quad (81)$$

$$= [\mathbf{U}_1 \mathbf{U}_2] \begin{bmatrix} \mathbf{S}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} [\mathbf{U}_1 \mathbf{U}_2]^T \quad (82)$$

$$= \mathbf{U}_1 \mathbf{S}_1 \mathbf{U}_1^T. \quad (83)$$

Of course, the eigen vectors with to zero eigen-values don't contribute to $\mathbf{K}$. However, they do contribute to $\mathbf{K} + \mathbf{I}\delta$ since the matrix is now full rank. Using properties of linear algebra and the spectral decomposition, the dependence on $\mathbf{U}_2$ can be removed since

$$\mathbf{K} + \mathbf{I}\delta = \mathbf{U}\mathbf{S}\mathbf{U}^T + \mathbf{U}\mathbf{U}^T\delta \quad (84)$$

$$= \mathbf{U}(\mathbf{S} + \mathbf{I}\delta)\mathbf{U}^T \quad (85)$$

$$= [\mathbf{U}_1 \mathbf{U}_2] \begin{bmatrix} \mathbf{S}_1 + \mathbf{I}_k\delta & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{n-k}\delta \end{bmatrix} [\mathbf{U}_1 \mathbf{U}_2]^T \quad (86)$$

$$= \mathbf{U}_1(\mathbf{S}_1 + \mathbf{I}_k\delta)\mathbf{U}_1^T + \mathbf{U}_2\mathbf{U}_2^T\delta \quad (87)$$

$$= \mathbf{U}_1(\mathbf{S}_1 + \mathbf{I}_k\delta)\mathbf{U}_1^T + (\mathbf{I} - \mathbf{U}_1\mathbf{U}_1^T)\delta. \quad (88)$$

Therefore, only the nonzero components of the spectral decomposition are needed. See additional details in our previous work (Hoffman and Roussos 2025) motivated by Lippert *et al.* (2011).

With this form of $\mathbf{K} + \mathbf{I}\delta$ using the low rank spectral decomposition, we can remove the dependence on $\mathbf{U}_2$ from the log-likelihood. The log determinant can be expressed as

$$\log |\mathbf{K} + \mathbf{I}\delta| = \sum_i \log(\mathbf{s}_i + \delta) \quad (89)$$

$$= \sum_{i=1}^r \log(\mathbf{s}_i + \delta) + (n - r) \log \delta. \quad (90)$$

The quadratic form can be simplified according to

$$(\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{K} + \mathbf{I}\delta)^{-1} (\mathbf{y} - \mathbf{X}\beta) = (\mathbf{y} - \mathbf{X}\beta)^T \mathbf{U} (\mathbf{S} + \mathbf{I}\delta)^{-1} \mathbf{U}^T (\mathbf{y} - \mathbf{X}\beta) \quad (91)$$

$$= (\mathbf{U}_1^T \mathbf{y} - \mathbf{U}_1^T \mathbf{X}\beta)^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} (\mathbf{U}_1^T \mathbf{y} - \mathbf{U}_1^T \mathbf{X}\beta) +$$

$$(\mathbf{U}_2^T \mathbf{y} - \mathbf{U}_2^T \mathbf{X}\beta)^T (\mathbf{I}_{n-k} \delta)^{-1} (\mathbf{U}_2^T \mathbf{y} - \mathbf{U}_2^T \mathbf{X}\beta)$$

$$= (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta)^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) + \quad (92)$$

$$\mathbf{U}_2 (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \mathbf{U}_2^T / \delta, \quad (93)$$

where $\tilde{\mathbf{y}} = \mathbf{U}_1^T \mathbf{y}$ and $\tilde{\mathbf{X}} = \mathbf{U}_1^T \mathbf{X}$.

Simplifying the second term gives

$$\mathbf{U}_2 (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \mathbf{U}_2^T / \delta = (\mathbf{I} - \mathbf{U}_1 \mathbf{U}_1^T) (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) (\mathbf{I} - \mathbf{U}_1 \mathbf{U}_1^T)^T / \delta \quad (94)$$

$$= (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) / \delta - \mathbf{U}_1 \mathbf{U}_1^T (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \mathbf{U}_1^T \mathbf{U}_1 / \delta$$

$$= (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) / \delta - \mathbf{U}_1^T (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta)^T (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta) \mathbf{U}_1 / \delta$$

$$= \mathbf{r}^T \mathbf{r} / \delta - \tilde{\mathbf{r}}^T \tilde{\mathbf{r}} / \delta, \quad (95)$$

where $\mathbf{r} = \mathbf{y} - \mathbf{X}\beta$ and $\tilde{\mathbf{r}} = \tilde{\mathbf{y}} - \tilde{\mathbf{X}}\beta$.

Putting this together gives

$$(\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{K} + \mathbf{I}\delta)^{-1} (\mathbf{y} - \mathbf{X}\beta) = \mathbf{r}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \mathbf{r} + \mathbf{r}^T \mathbf{r} / \delta - \tilde{\mathbf{r}}^T \tilde{\mathbf{r}} / \delta, \quad (96)$$

so the log-likelihood is then

$$\ell(\beta, \sigma_g^2, \delta) = -\frac{1}{2} \left(n \log(2\pi\sigma_g^2) + \sum_{i=1}^r \log(\mathbf{s}_i + \delta) + (n - k) \log \delta \right. \quad (97)$$

$$\left. + \frac{1}{\sigma_g^2} \left(\mathbf{r}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \mathbf{r} + \mathbf{r}^T \mathbf{r} / \delta - \tilde{\mathbf{r}}^T \tilde{\mathbf{r}} / \delta \right) \right).$$

This form of the log-likelihood can now be used for parameter estimation as follows.

Regression coefficients

Taking the gradient of the log-likelihood with respect to β , and setting to equal to zero gives the coefficient estimate

$$\hat{\beta} = \left(\tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{X}} + \mathbf{X}^T \mathbf{X} / \delta - \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} / \delta \right)^{-1} \left(\tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{y}} + \mathbf{X}^T \mathbf{y} / \delta - \tilde{\mathbf{X}}^T \tilde{\mathbf{y}} / \delta \right). \quad (98)$$

Instead of evaluating every term here for each value of δ , this formula has terms that can be precomputed. Letting

$$\mathbf{\Gamma}_{X,X} = \mathbf{X}^T \mathbf{X} - \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \quad (99)$$

$$\mathbf{\Gamma}_{X,y} = \mathbf{X}^T \mathbf{y} - \tilde{\mathbf{X}}^T \tilde{\mathbf{y}}, \quad (100)$$

the coefficient estimate simplifies to

$$\hat{\beta} = \left(\tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{X}} + \mathbf{\Gamma}_{X,X} / \delta \right)^{-1} \left(\tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{y}} + \mathbf{\Gamma}_{X,y} / \delta \right). \quad (101)$$

Similarly,

$$\text{var}(\hat{\beta}) = \left(\tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{X}} + \mathbf{\Gamma}_{X,X} / \delta \right)^{-1} \hat{\sigma}_a^2. \quad (102)$$

Variance parameter σ_g^2

Given $\hat{\beta}$ and a fixed value of δ , find the maximum likelihood estimate of σ_g^2 by setting the gradient to zero. This gives the estimate

$$\hat{\sigma}_a^2 = \frac{1}{n} \left(\mathbf{r}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \mathbf{r} + \mathbf{r}^T \mathbf{r} / \delta - \tilde{\mathbf{r}}^T \tilde{\mathbf{r}} / \delta \right). \quad (103)$$

Variance ratio δ

For a given value of δ , compute $\hat{\beta}(\delta)$ and $\hat{\sigma}_a^2(\delta)$ and estimate δ according to

$$\hat{\delta} = \underset{\delta}{\text{argmax}} \ell \left(\hat{\beta}(\delta), \hat{\sigma}_a^2(\delta), \delta \right). \quad (104)$$

Because $\hat{\beta}(\delta)$ and $\hat{\sigma}_a^2(\delta)$ have closed form solutions, we can plug the estimates back into the log-likelihood. Given a value of δ , first evaluate $\hat{\beta}$. Next evaluate $\mathbf{r}$ and $\tilde{\mathbf{r}}$, noting that the coefficient estimate only enters the log-likelihood through these two residual terms. These terms are then used to estimate $\hat{\sigma}_a^2$. Plugging this back into the log-likelihood gives

$$\ell \left(\hat{\beta}(\delta), \hat{\sigma}_a^2(\delta), \delta \right) = -\frac{1}{2} \left(n \log(2\pi \hat{\sigma}_g^2) + \sum_{i=1}^r \log(\mathbf{s}_i + \delta) + (n - r) \log \delta + n \right). \quad (105)$$

11.3. Satterthwaite approximate degrees of freedom

Consider testing the hypothesis $\mathbf{L}\beta = 0$ with the Wald statistic

$$t = \frac{\mathbf{L}\beta}{\sqrt{\mathbf{L}\hat{\Sigma}_\beta \mathbf{L}^T}}. \quad (106)$$

Let $\omega = (\sigma_g^2, \sigma_e^2)$ be the vector of variance parameters, and function $f(\omega) = \mathbf{L}\hat{\Sigma}_\beta(\omega)\mathbf{L}^T$ where $\hat{\Sigma}_\beta(\omega)$ is variance-covariance matrix of the regression coefficients as a function of variance

parameters. Following previous work (Fai and Cornelius 1996; Kuznetsova *et al.* 2017; Christensen 2020), the null distribution of the test statistic t can be approximated with a Student's t distribution with the degrees of freedom

$$\hat{\nu} = \frac{2f(\hat{\omega})^2}{\text{var}[f(\hat{\omega})]}. \quad (107)$$

While $f(\hat{\omega})$ can be evaluated from the data, the value of the denominator cannot be obtained directly. Instead, we use the delta method to approximate the variance as

$$\text{var}[f(\hat{\omega})] \approx \nabla f(\hat{\omega})^T \text{var}(\hat{\omega}) \nabla f(\hat{\omega}). \quad (108)$$

We now consider each term in this approximation.

Gradient term

The term $\nabla f(\hat{\omega})$ is the matrix of gradients of $f(\cdot)$ with respect to ω evaluated at $\hat{\omega} = (\hat{\sigma}_g^2, \hat{\sigma}_e^2)$. Plugging in the variance of the coefficient estimates from Eqn (70),

$$f(\omega) = \mathbf{L} \left(\tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}} \right)^{-1} \mathbf{L}^T \hat{\sigma}_a^2. \quad (109)$$

so that

$$\frac{\partial f}{\partial \sigma_g^2} = \mathbf{L}^T \mathbf{A}^{-1} \mathbf{L} - \delta \mathbf{L}^T \mathbf{A}^{-1} \mathbf{B} \mathbf{A}^{-1} \mathbf{L} \quad (110)$$

$$\frac{\partial f}{\partial \sigma_e^2} = \mathbf{L}^T \mathbf{A}^{-1} \mathbf{B} \mathbf{A}^{-1} \mathbf{L}, \quad (111)$$

where

$$\delta = \sigma_e^2 / \sigma_g^2 \quad (112)$$

$$A(\delta) = \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-1} \tilde{\mathbf{X}} \quad (113)$$

$$= \tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-1} \tilde{\mathbf{X}} + \mathbf{\Gamma}_{X,X} / \delta \quad (114)$$

$$B(\delta) = \tilde{\mathbf{X}}^T (\mathbf{S} + \mathbf{I}\delta)^{-2} \tilde{\mathbf{X}} \quad (115)$$

$$= \tilde{\mathbf{X}}^T (\mathbf{S}_1 + \mathbf{I}_k \delta)^{-2} \tilde{\mathbf{X}} + \mathbf{\Gamma}_{X,X} / \delta^2. \quad (116)$$

Note that $A(\delta)$, $B(\delta)$ can be evaluated efficiently by recycling terms already computed in the model fit, and can be computed before $\mathbf{L}$ is specified.

Hessian term

The term $\text{var}(\hat{\omega})$ is the variance-covariance matrix of the variance parameters, and can be obtained from the Hessian of the log-likelihood function, $H_{\hat{\omega}}$, according to

$$\text{var}(\hat{\omega}) = -H_{\hat{\omega}}^{-1}. \quad (117)$$

Evaluating the partial derivatives of Eqn (105) with respect to σ_g^2 and σ_e^2 gives

$$H_{\hat{\omega}} = \nabla^2 f(\sigma_g^2, \sigma_e^2) \quad (118)$$

$$= \frac{1}{\sigma_g^4} \begin{bmatrix} n - 2a(\delta) + \delta^2 b(\delta) & a(\delta) - b(\delta) \\ a(\delta) - b(\delta) & b(\delta) \end{bmatrix} \quad (119)$$

where

$$\delta = \sigma_e^2 / \sigma_g^2 \quad (120)$$

$$a(\delta) = \sum_{i=1}^r \frac{1}{s_i + \delta} + \frac{n-r}{\delta} \quad (121)$$

$$b(\delta) = \sum_{i=1}^r \frac{1}{(s_i + \delta)^2} + \frac{n-r}{\delta^2} \quad (122)$$

This is easily evaluated in $\mathcal{O}(r)$ time.

Returning to evaluating the approximate degrees of freedom, we see that $\hat{\nu}$ can be evaluated efficiently given the parameter estimates. In addition, the terms $H_{\hat{\omega}}$, $A(\hat{\delta})$ and $B(\hat{\delta})$ can be evaluated a single time, and recycled for multiple contrast matrices $\mathbf{L}$.

Comparison to other packages

We note that our estimates of $\hat{\nu}$ match the values from **lmerTest** and **glmmTMB** for Gaussian models. Estimating $\hat{\nu}$ with **fastglmm** takes a negligible amount of time since both Hessian and gradients are evaluated in closed form given the parameter estimates. Meanwhile, applying the Satterthwaite method is computationally demanding with existing packages since **lmerTest** and **glmmTMB** estimate the Hessian and gradients numerically.

For GLMMs, the value of $\hat{\nu}$ differs slightly from the value produced by **glmmTMB**. This is likely due to the fact that **fastglmm** estimates parameters using penalized quasi-likelihood, while **glmmTMB** uses a Laplace approximation.

Affiliation:

Gabriel E. Hoffman

Center for Disease Neurogenomics

Department of Psychiatry

Department of Genetics and Genomic Sciences

Icahn School of Medicine at Mount Sinai

1470 Madison Ave.

New York, NY 10029, USA

E-mail: gabriel.hoffman@mssm.edu

Mental Illness Research, Education and Clinical Centers

James J. Peters VA Medical Center

130 W Kingsbridge Rd.

Bronx, NY 10468, USA